Emergency Pathfinding Project (Germany)

Using OSMnx, NetworkX, Dijkstra and A*

```
!pip install osmnx networkx matplotlib
```

```
Collecting osmnx
  Downloading osmnx-2.1.0-py3-none-any.whl.metadata (4.7 kB)
Requirement already satisfied: networkx in /usr/local/lib/python3.12/dist-packages (3.6.1)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: geopandas>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from osmnx) (1.1.3)
Requirement already satisfied: numpy>=1.24 in /usr/local/lib/python3.12/dist-packages (from osmnx) (2.0.2)
Requirement already satisfied: pandas>=1.5 in /usr/local/lib/python3.12/dist-packages (from osmnx) (2.2.2)
Requirement already satisfied: requests>=2.30 in /usr/local/lib/python3.12/dist-packages (from osmnx) (2.32.4)
Requirement already satisfied: shapely>=2.0 in /usr/local/lib/python3.12/dist-packages (from osmnx) (2.1.2)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.62.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: pyogrio>=0.7.2 in /usr/local/lib/python3.12/dist-packages (from geopandas>=1.0.1->osmnx) (0.12.1)
Requirement already satisfied: pyproj>=3.5.0 in /usr/local/lib/python3.12/dist-packages (from geopandas>=1.0.1->osmnx) (3.7.2)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.5->osmnx) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.5->osmnx) (2025.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (1.17
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.30->osmnx)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.30->osmnx) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.30->osmnx) (2.5.0
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.30->osmnx) (2026.
Downloading osmnx-2.1.0-py3-none-any.whl (104 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 104.4/104.4 kB 2.0 MB/s eta 0:00:00
Installing collected packages: osmnx
Successfully installed osmnx-2.1.0
```

```
import osmnx as ox
import networkx as nx
import matplotlib.pyplot as plt
import random
```

```
city = "Heidelberg, Germany"
G = ox.graph_from_place(city, network_type='drive')
print("Graph loaded with", len(G.nodes), "nodes")
```

```
Graph loaded with 2857 nodes
```

```
tags = {"amenity": "hospital"}
hospitals = ox.features_from_place(city, tags)
hospitals = hospitals[hospitals.geometry.type == "Point"]
print("Hospitals found:", len(hospitals))
```

```
Hospitals found: 6
```

```
G = G.to_undirected()
```

```
hospital_nodes = []

for _, row in hospitals.iterrows():
    try:
        x = row.geometry.x
        y = row.geometry.y
        nearest_node = ox.distance.nearest_nodes(G, x, y)
        hospital_nodes.append(nearest_node)
    except:
        continue

hospital_nodes = list(set(hospital_nodes))
print("Mapped hospital nodes:", len(hospital_nodes))
```

```
Mapped hospital nodes: 4
```

```
start_node = random.choice(list(G.nodes))
print("Start node:", start_node)
```

```
Start node: 410654152
```

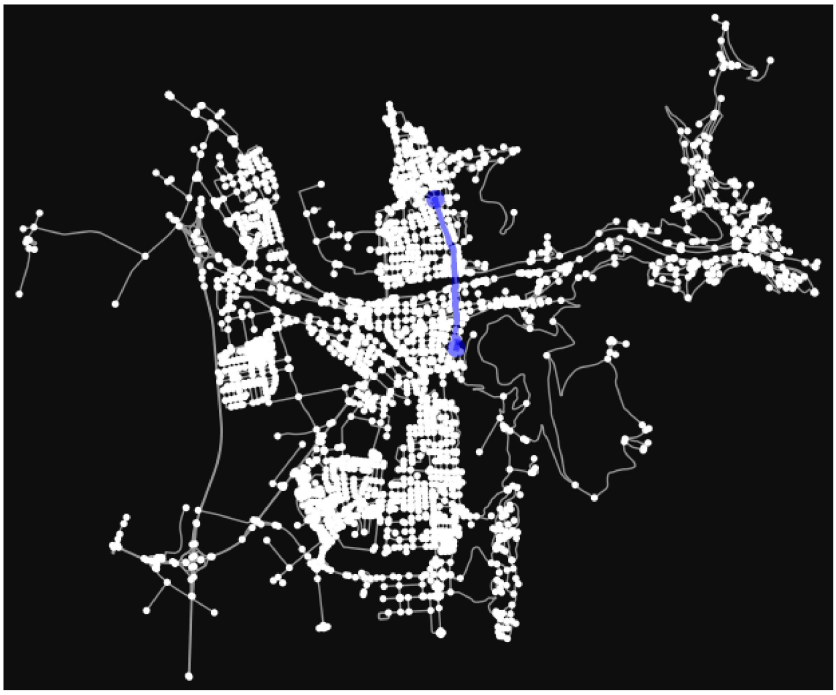
```
shortest_path_dijkstra = None
min_dist = float("inf")

for hospital in hospital_nodes:
    try:
        length = nx.shortest_path_length(G, start_node, hospital, weight="length")
        if length < min_dist:
            min_dist = length
            shortest_path_dijkstra = nx.shortest_path(G, start_node, hospital, weight="length")
    except:
        continue

print("Dijkstra distance:", min_dist)
```

```
Dijkstra distance: 2646.587962735463
```

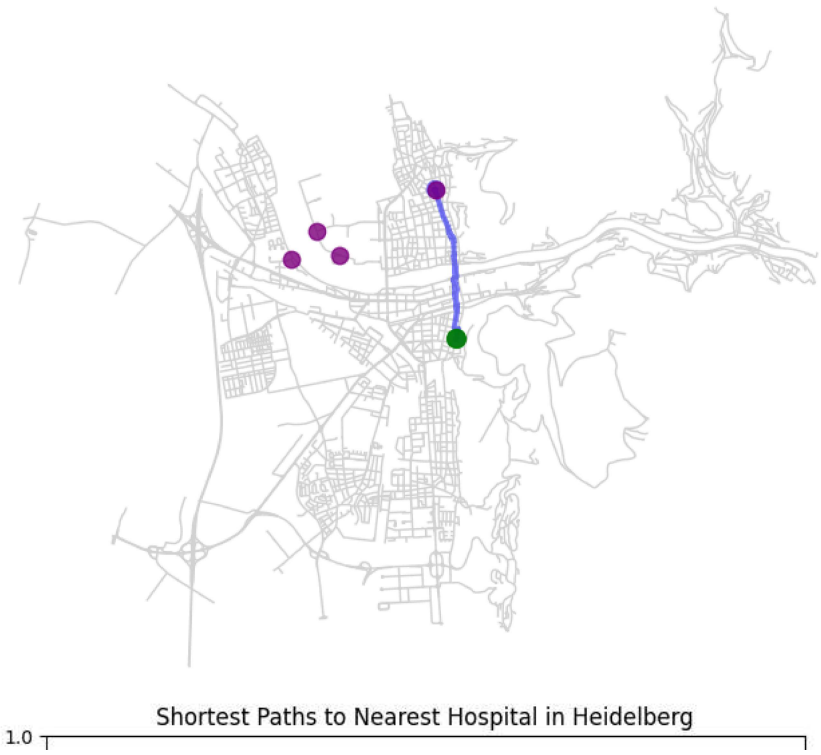
```
def heuristic(u, v):
    x1, y1 = G.nodes[u]['x'], G.nodes[u]['y']
    x2, y2 = G.nodes[v]['x'], G.nodes[v]['y']
    return ((x1 - x2)**2 + (y1 - y2)**2)**0.5

shortest_path_astar = None
min_dist_astar = float("inf")

for hospital in hospital_nodes:
    try:
        path = nx.astar_path(G, start_node, hospital, heuristic=heuristic, weight="length")
        length = nx.path_weight(G, path, weight="length")
        if length < min_dist_astar:
            min_dist_astar = length
            shortest_path_astar = path
    except:
        continue

print("A* distance:", min_dist_astar)
```

```
A* distance: 2646.587962735463
```

```
fig, ax = ox.plot_graph(G, show=False, close=False)

if shortest_path_dijkstra:
    ox.plot_graph_route(G, shortest_path_dijkstra, ax=ax, route_color="blue", route_linewidth=3, node_size=0)

if shortest_path_astar:
    ox.plot_graph_route(G, shortest_path_astar, ax=ax, route_color="red", route_linewidth=3, node_size=0)

plt.show()
```



```
fig, ax = ox.plot_graph(G, show=False, close=False, bgcolor='white', edge_color='lightgray', node_size=0)

# Get coordinates for the start node
start_node_x, start_node_y = G.nodes[start_node]['x'], G.nodes[start_node]['y']
# Plot the start node
ax.scatter(start_node_x, start_node_y, color='green', s=100, zorder=5, alpha=0.9, label='Start Node')

# Get coordinates for the hospital nodes
hospital_xs = [G.nodes[node]['x'] for node in hospital_nodes]
hospital_ys = [G.nodes[node]['y'] for node in hospital_nodes]
# Plot the hospital nodes
ax.scatter(hospital_xs, hospital_ys, color='purple', s=80, zorder=4, alpha=0.8, label='Hospital Nodes')

if shortest_path_dijkstra:
    ox.plot_graph_route(G, shortest_path_dijkstra, ax=ax, route_color="blue", route_linewidth=3, node_size=0, label='Dijkstra Pa

if shortest_path_astar:
    ox.plot_graph_route(G, shortest_path_astar, ax=ax, route_color="red", route_linewidth=3, node_size=0, label='A* Path')

# Add a legend to differentiate paths and nodes
ax.legend()
plt.title("Shortest Paths to Nearest Hospital in Heidelberg")
plt.tight_layout()
plt.show()
```


Shortest Paths to Nearest Hospital in Heidelberg
1.0